

Repository Insight Analysis of NumPy Development Activity

Research Question 1: How does development activity evolve over time?

Caddinate name : Duja Shelbai

Contents

Introduction.....	3
Methodology and Data Extraction.....	3
Data Cleaning Decisions.....	3
Results and Visualizations	4
Interpretation and Discussion	5
Validity, Limitations, and Follow-up Research Question	6
6. Conclusion	6

Introduction

This report presents a structured empirical analysis of the GitHub repository `numpy/numpy`. The aim of the analysis is to understand how development activity has evolved across the lifetime of the project. NumPy is a long-running open-source scientific computing library, making it a suitable case study for observing software evolution over an extended period.

The selected research question is:

RQ1 — How does development activity evolve over time?

To answer this question, the analysis focuses on monthly commit counts and code churn.

Commit count is used as an indicator of development frequency, while code **churn** is measured as the total number of lines added and removed. Together, these measures help show how large or intensive those changes were.

Methodology and Data Extraction

The analysis was conducted using Git and Python. First, the NumPy repository was cloned from GitHub. Git log data was then extracted using the `--numstat` option, which provides file-level line additions and deletions for each commit. Merge commits were excluded using `--no-merges` to reduce duplication caused by integration commits. The extracted fields included commit hash, commit date, author name, author email, lines added, lines removed, and number of changed files. The raw Git log was saved as `raw_git_log.txt` and then parsed using a Python script. The parsed commit-level data was aggregated into monthly observations.

The final cleaned dataset contains 292 monthly observations, covering the period from December 2001 to May 2026. In total, 31,075 non-merge commits were analyzed. Across the full period, the repository recorded 3,092,595 lines added, 2,728,205 lines removed, and total code churn of 5,820,800 lines.

Data Cleaning Decisions

Several data cleaning decisions were made to ensure the analysis was consistent and reproducible. Dates were converted into monthly periods, allowing all commits within the same calendar month to be grouped together. For each month, the script calculated the total number of commits, total lines added, total lines removed, total churn, and total number of changed files. Binary file changes were excluded from churn calculations because Git reports binary changes using a dash symbol rather than numeric line additions or deletions. Merge commits were also excluded to avoid counting integration activity that may duplicate work already represented in individual commits.

Code churn was calculated as:

total churn = lines added + lines removed

This metric captures the volume of line-level change in the repository, but it does not directly measure the semantic importance or quality of those changes.

Results and Visualizations

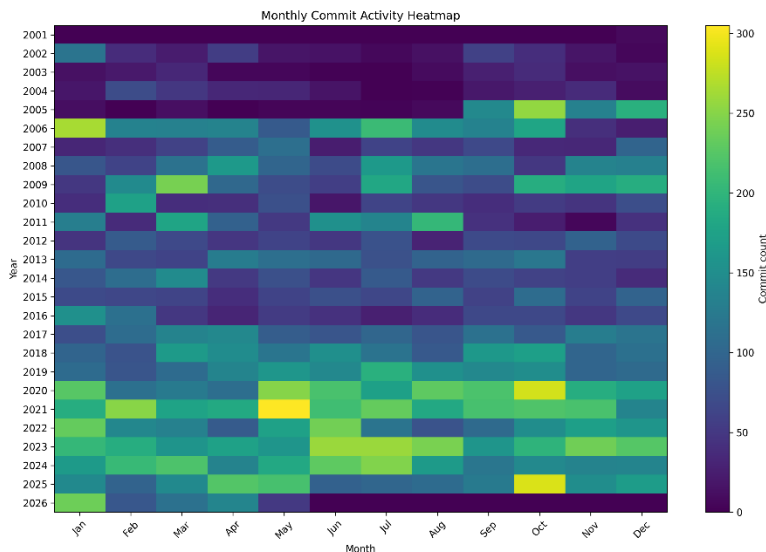


Figure 1. Monthly Commit Count and Total Code Churn Over Time

Figure 1 shows the monthly commit count and total code churn across the lifetime of the NumPy repository. The results show that commit activity became more sustained over time. Earlier years contain several periods of lower or irregular activity, while later years show more continuous development. The figure also shows that commit count and code churn do not always increase at the same time. Some months have very high churn even when the number of commits is not the highest. This suggests that a smaller number of commits can sometimes introduce very large line-level changes, possibly due to refactoring, restructuring, or large feature integrations.

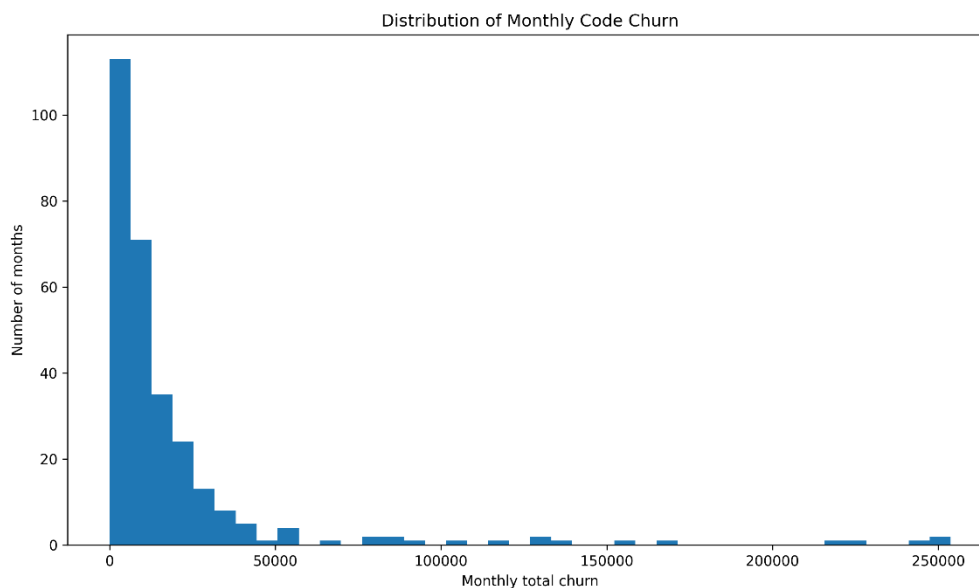


Figure 2. Distribution of Monthly Code Churn

Figure 2 shows the distribution of monthly code churn. The distribution is strongly right-skewed. Most months have relatively moderate churn, while a small number of months have extremely high churn values. This indicates that NumPy development is usually incremental, but occasionally experiences large bursts of code change. These high-churn months may correspond to major refactoring, large-scale cleanup, migration work, or substantial feature development.

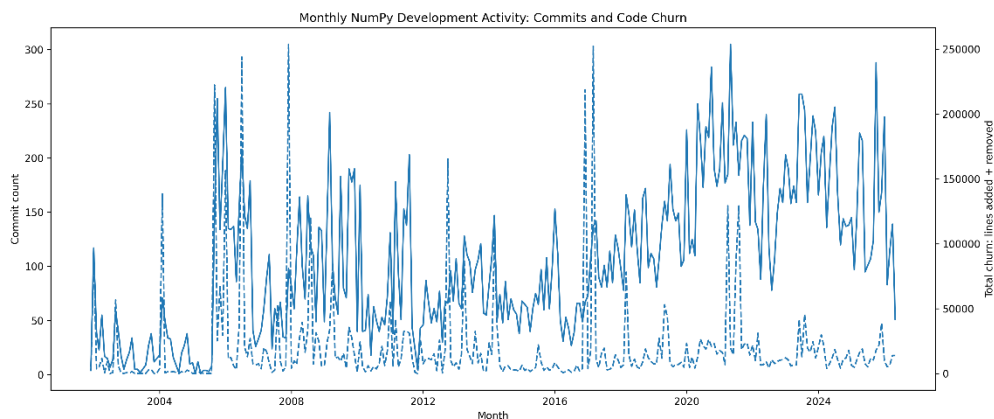


Figure 3. Monthly Commit Activity Heatmap

Figure 3 shows monthly commit activity as a year-month heatmap. Darker cells represent lower commit activity, while brighter cells represent higher commit activity. The heatmap shows that activity became more consistent in later years, especially after approximately 2018. The heatmap also makes it easier to identify concentrated periods of high activity, such as 2021 and 2025. However, 2026 should be interpreted carefully because the dataset only includes commits up to May 2026, meaning that 2026 is an incomplete year.

Interpretation and Discussion

The results suggest that NumPy's development activity has increased and become more sustained over time. Across the full dataset, the average monthly commit count was **106.42 commits**, and the average monthly churn was **19,934.25 lines**.

A key finding is that commit count and code churn measure different aspects of development. Commit count reflects how frequently changes are made, while churn reflects the size of those changes. For example, **May 2021** had the highest number of commits, but its churn was **19,755 lines**, which is close to the average monthly churn. In contrast, **December 2007** had fewer commits but much higher churn, with **253,777 lines changed**.

This means that high development activity does not always mean large code changes. Some months may involve many small commits, such as bug fixes, maintenance, or incremental improvements. Other months may involve fewer but larger commits, possibly related to refactoring, cleanup, migration, or architectural changes.

Overall, the results show that NumPy has evolved from a more irregular early-stage project into a mature open-source project with sustained development activity. At the same time, the project still experienced occasional large churn spikes, suggesting important technical transitions at specific points in its history

Validity, Limitations, and Follow-up Research Question

There are several limitations to this analysis. For example, Git history only captures committed code changes. It does not capture discussions, issue reports, pull request reviews, design decisions, or community coordination.

Also, code churn does not always represent meaningful development. Large line changes may be caused by formatting, generated files, file movement, or refactoring rather than new functionality. Therefore, churn should be interpreted as the volume of change, not necessarily the importance or quality of change. And merge commits were excluded to reduce duplicated integration activity. This improves the clarity of the analysis, but it may also remove some information about collaboration and integration workflow.

The analysis was conducted on the repository state at the time of cloning. Since NumPy is still actively developed, future commits may change the results. Also, 2026 is incomplete because the available data only reaches May 2026.

A meaningful follow-up research question would be:

How do periods of high code churn relate to release preparation, bug fixing, major refactoring, or architectural changes in NumPy?

This follow-up question would connect the quantitative results to qualitative project events. Future work could combine Git history with release notes, issue labels, pull request discussions, and file-level analysis to explain the causes of major churn spikes.

6. Conclusion

In conclusion, this study shows that NumPy's development activity has become more sustained over time. Commit activity has generally increased in recent years, while code churn shows a more

uneven pattern with occasional large spikes. The main finding is that commit count and code churn should be interpreted together: commit count shows the frequency of development, while churn shows the size and intensity of changes. This combined view provides a clearer understanding of how a mature open-source scientific software project evolves over time.